

The background of the slide features the digits 1 through 9 and 0 arranged in a circular pattern. Each digit is a different color and has a distinct texture: 1 is yellow with a grid pattern, 2 is purple with a grid pattern, 3 is blue with a grid pattern, 4 is orange with a grid pattern, 5 is green with a grid pattern, 6 is green with a grid pattern, 7 is red with a grid pattern, 8 is yellow with a grid pattern, 9 is purple with a grid pattern, and 0 is blue with a grid pattern. The text is overlaid on a semi-transparent grey rectangle in the center.

**Reconhecimento de padrão de
caracteres numéricos com
Rede Neural Artificial
e algoritmo *Backpropagation***

profº Mauricio Conceição Mario

```
temp.py X busca_binária-V.py X busca_binária-IV.py X padrão_caracteres_numéricos.py X
3 Created on Wed Dec 30 21:06:04 2020
4
5 @author: cmari
6 """
7 from linear_algebra import dot
8 import math, random
9
10 def sigmoid(t):
11     return 1 / (1 + math.exp(-t))
12
13 def neurônio_MCP(pesos, entradas):
14     uk = dot(pesos, entradas)
15     return sigmoid(uk)
16
17 def feed_forward(rede, vetor_entrada):
18     """recebe uma rede neural (representada como uma lista de listas de listas de pesos)
19     e retorna a saída da propagação direta da entrada"""
20
21     vetor_saída = []
22
23     for ponteiro in rede:
24
25         entrada_com_bias = vetor_entrada + [1] # adiciona bias ao vetor de entrada
26         saída = [neurônio_MCP(neurônio, entrada_com_bias) # calcula a saída
27                 for neurônio in ponteiro] # para a camada de rede
28         vetor_saída.append(saída) # adiciona ao vetor de saída
29
30         # a entrada para a próxima camada é a saída desta etapa
31         vetor_entrada = saída
32
33     return vetor_saída
34
```




35

Cálculo dos deltas para atualização dos pesos sinápticos:

36 alpha = 0.99

37 def backpropagation(rede_neural, vetor_entrada, vetor_saída):

derivada da sigmóide unipolar na camada de saída * erro

38 # feed_forward calcula a saída dos neurônios usando sigmóide (retorno do algoritmo Backpropagation)

39 saídas_intermediárias, saídas_neurônios = feed_forward(rede_neural, vetor_entrada)

40

41 deltas_saída = [saída * (1 - saída) * (saída - vetor_saída[i]) * alpha

42 for i, saída in enumerate(saídas_neurônios)]

Atualização dos pesos da camada de saída = delta camada saída * sigmóide

44 # ajuste dos pesos sinápticos para camada de saída (rede_neural[-1])

45 for i, neurônio_saída in enumerate(rede_neural[-1]):

derivada da sigmóide unipolar na
camada intermediária * produto dos
deltas de camada de rede de saída
(retorno do algoritmo
Backpropagation)

46 for j, saída_intermediária in enumerate(saídas_intermediárias + [1]):

47 neurônio_saída[j] -= deltas_saída[i] * saída_intermediária

48

49 # 0.5 * alpha * (1 + output) * (1 - output) calculo da derivada da sigmóide

50 # retro-propagação do erro para camadas intermediárias

51 deltas_intermediários = [saída_intermediária * (1 - saída_intermediária) *

52 dot(deltas_saída, [n[i] for n in rede_neural[-1]])

53 for i, saída_intermediária in enumerate(saídas_intermediárias)]

Atualização dos pesos da camada intermediária = delta camada intermediária * entradas

55 # ajuste dos pesos sinápticos para camadas intermediárias (network[0])

56 for i, neurônio_intermediário in enumerate(rede_neural[0]):

cada um dos vetores com caracteres numéricos

57 for j, entrada in enumerate(vetor_entrada + [1]):

+ entrada de bias dos neurônios da camada intermediária

58 neurônio_intermediário[j] -= deltas_intermediários[i] * entrada

59

60 if __name__ == "__main__":

Criação de variável __name__ que armazena nome de módulo (= arquivo.py) importado; se a condição if é satisfeita executa o que está indentado a ela. Configura __name__ para "__main__" do módulo TextoUtil.py que possui funções para manipulação de strings. (Summerfield, 2013)

61 lista_treino = [

62 """11111

63 1...1

64 1...1

65 1...1

66 11111""",

67

68

função enumerate desmembra os elementos de rede_neural para os, correspondentes pesos sinápticos dos segmentos de entrada e saída da rede.

verificar

identação

```

68
69     """..1..
70         ..1..
71         ..1..
72         ..1..
73         ..1.. """ ,
74
75     """11111
76         ....1
77         11111
78         1.....
79         11111 """ ,
80
81     """11111
82         ....1
83         11111
84         ....1
85         11111 """ ,
86
87     """1...1
88         1...1
89         11111
90         ....1
91         ....1 """ ,
92
93     """11111
94         1.....
95         11111
96         ....1
97         11111 """ ,
98

```

verificar
↔
identação

vetores de entrada: INPUTS

```

[[1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1], [0, 0, 1,
0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0], [1, 1, 1, 1, 1, 0, 0, 0, 0,
0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1], [1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0,
0, 1, 0, 0, 0, 0, 1], [1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1,
1, 1], [1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1], [1, 1,
1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1], [1, 1, 1, 1, 1, 1, 0,
0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1,
1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1]]

```

```

98
99     """11111
100        1....
101        11111
102        1...1
103        11111""" ,
104
105     """11111
106        ....1
107        ....1
108        ....1
109        ....1""" ,
110
111     """11111
112        1...1
113        11111
114        1...1
115        11111""" ,
116
117     """11111
118        1...1
119        11111
120        ....1
121        11111""" ]
122
123 def make_digit(dados_treino):
124     return [1 if c == '1' else 0
125             for row in dados_treino.split("\n")
126             for c in row.strip()]
127
128 caracteres_numéricos = list(map(make_digit, lista_treino))
129 print("\nvetores de entrada: INPUTS\n ", caracteres_numéricos)
130

```

verificar

identação

entradas = lista dos caracteres
formatados com 1 se "1" e
0 se ".".



temp.py ✕ busca_binária-V.py ✕ busca_binária-IV.py ✕ padrão_caracteres_numéricos.py ✕

```

127
128 caracteres_numéricos = list(map(make_digit, lista_treino))
129 print("\nvetores de entrada: INPUTS\n ", caracteres_numéricos)
130
131 saídas = [[1 if i == j else 0 for i in range(10)]
132           for j in range(10)]
133
134 random.seed(0) # pode-se utilizar valores repetidos a partir dos randômicos
135 dimensão_entrada = 25 # dimensão dos vetores relacionados às 10 entradas
136 neurônios_ocultos = 5 # quantidade de neurônios na camada intermediária
137 dimensão_saída = 10 # 10 saídas, cada uma relacionada à uma entrada
138
139 camada_intermediária = [[random.random() for __ in range(dimensão_entrada + 1)]
140                          for __ in range(neurônios_ocultos)]
141
142 camada_saída = [[random.random() for __ in range(neurônios_ocultos + 1)]
143                for __ in range(dimensão_saída)]
144
145 rede_neural = [camada_intermediária, camada_saída]
146
147 # 10000 CICLOS DE TREINAMENTO
148 for __ in range(10000):
149     for vetor_entrada, vetor_saída in zip(caracteres_numéricos, saídas):
150         backpropagation(rede_neural, vetor_entrada, vetor_saída)
151
152 def predict(números):
153     return feed_forward(rede_neural, números)[-1]
154 # TESTE DAS ENTRADAS TREINADAS
155 for i, números in enumerate(caracteres_numéricos):
156     saídas_rede = predict(números)
157     print(i, [round(p,2) for p in saídas_rede])#resultado dos neurônios de saída com 2 casas decimais

```

saídas = 10 saídas equivalentes a cada um dos caracteres, supondo que os mesmos estejam ordenados: por exemplo, se deseje **[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]** na saída para o 5º caractere numérico (4).

```
158 # TESTE DE DÍGITOS NUMÉRICOS QUE NÃO FORAM TREINADOS
159 print(""".@@@.
160 ...@@
161 ..@@.
162 ...@@
163 .@@@.""")
164 print([round(x, 2) for x in
165       predict( [0,1,1,1,0,      # .@@@.
166                0,0,0,1,1,      # ...@@
167                0,0,1,1,0,      # ..@@.
168                0,0,0,1,1,      # ...@@
169                0,1,1,1,0]]) # .@@@.
170 print("interpreta como dígito 3 com vestígios do dígito 9")
171
172 print(""".@@@.
173 @..@@
174 .@@@.
175 @..@@
176 .@@@.""")
177 print([round(x, 2) for x in
178       predict( [0,1,1,1,0,      # .@@@.
179                1,0,0,1,1,      # @..@@
180                0,1,1,1,0,      # .@@@.
181                1,0,0,1,1,      # @..@@
182                0,1,1,1,0]]) # .@@@.
183 print("interpreta como possível dígito 8 com vestígios do 9")
184
185
```

verificar
identificação

 Console 1/A ✕

```

0 [0.99, 0.0, 0.0, 0.0, 0.0, 0.0, 0.02, 0.01, 0.03, 0.0]
1 [0.0, 0.97, 0.03, 0.0, 0.03, 0.0, 0.0, 0.0, 0.0, 0.0]
2 [0.0, 0.02, 0.96, 0.02, 0.0, 0.02, 0.0, 0.0, 0.0, 0.0]
3 [0.0, 0.0, 0.03, 0.96, 0.0, 0.0, 0.0, 0.01, 0.0, 0.03]
4 [0.0, 0.02, 0.0, 0.0, 0.96, 0.0, 0.02, 0.0, 0.0, 0.0]
5 [0.0, 0.0, 0.02, 0.0, 0.0, 0.97, 0.02, 0.0, 0.02, 0.0]
6 [0.0, 0.0, 0.0, 0.0, 0.02, 0.02, 0.97, 0.0, 0.0, 0.0]
7 [0.01, 0.01, 0.0, 0.01, 0.0, 0.0, 0.0, 0.99, 0.0, 0.0]
8 [0.01, 0.0, 0.0, 0.0, 0.0, 0.02, 0.0, 0.0, 0.96, 0.03]
9 [0.0, 0.0, 0.0, 0.03, 0.0, 0.0, 0.0, 0.0, 0.03, 0.96]

```

*visualização
das saídas para
os caracteres
treinados*



```
.@@@.
```

```
...@@
```

```
..@@.
```

```
...@@
```

```
.@@@.
```

```
[0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.19, 0.0, 0.98]
```

```
interpreta como dígito 3 com vestígios do dígito 9
```

```
.@@@.
```

```
@..@@
```

```
.@@@.
```

```
@..@@
```

```
.@@@.
```

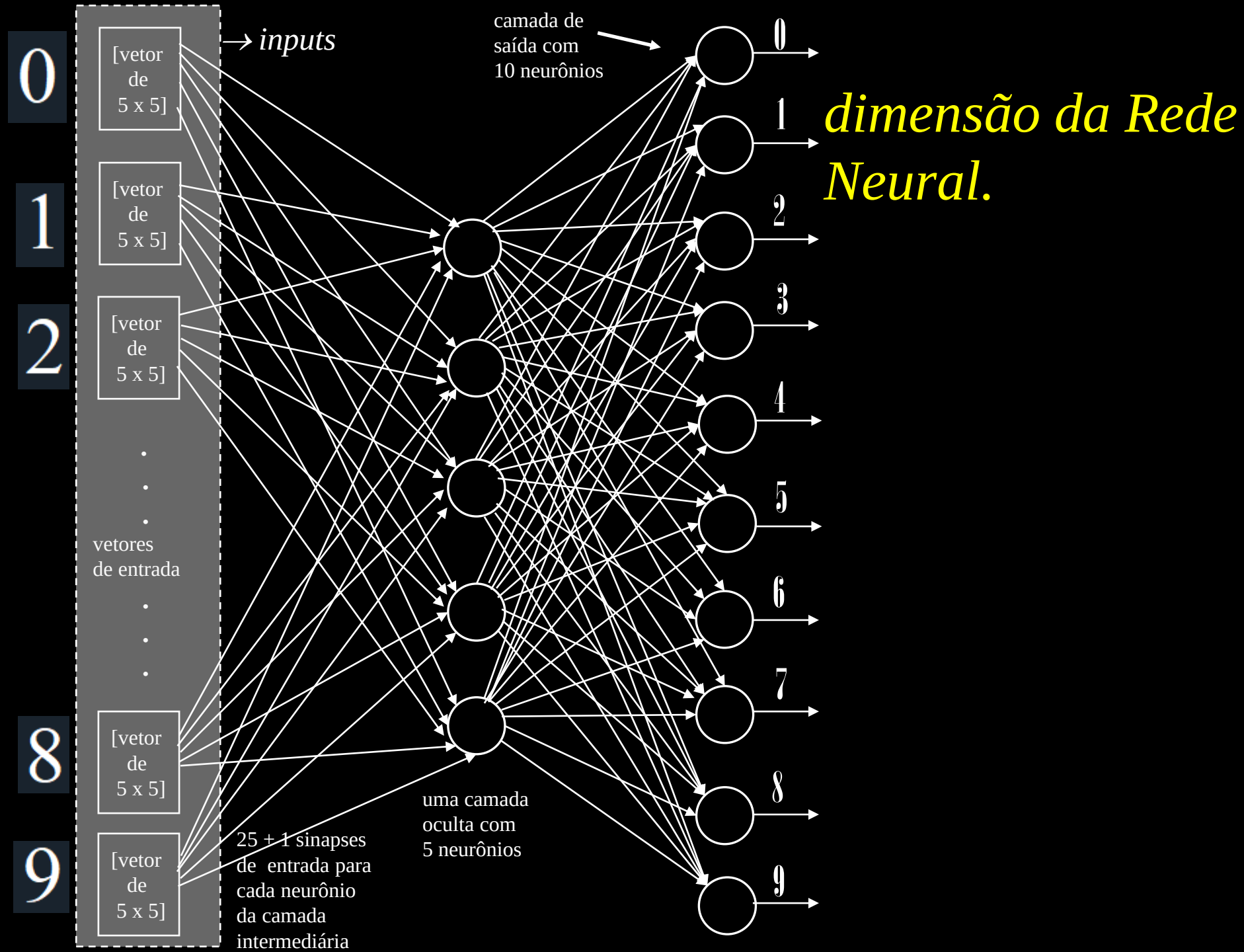
```
[0.01, 0.0, 0.0, 0.0, 0.0, 0.03, 0.0, 0.0, 1.0, 0.88]
```

```
interpreta como possível dígito 8 com vestígios do 9
```



*resultado dos
testes de
generalização
para os
caracteres 3 e 8*

Rede Neural



Referências Bibliográficas

- *Python para Análise de Dados*
Tratamento de Dados com *Pandas*, *Numpy* e *Ipython*
Wes McKinney - O'REILLY
editora **novatec** – São Paulo – 2020
- *Data Science do Zero*
Primeiras Regras com o *Python*
Joel Grus - O'REILLY
editora **Alta Books** – Rio de Janeiro – 2016
- *Programação em Python 3*
– *Uma Introdução Completa à Linguagem Python*
Mark Summerfield
Biblioteca do Programador – editora Alta Books - 2013

Referências Bibliográficas

- Braga AP, Carvalho APLF, Ludermir TB. *Redes Neurais Artificiais: teoria e aplicações*. Livros Técnicos e Científicos, Rio de Janeiro – RJ; 2007.
- Haykin S. *Neural Networks – A Comprehensive Foundation*. Prentice-Hall; 1994.
- Haykin S. *Redes Neurais – Princípios e prática*. 2a ed.. Porto Alegre: Bookman; 2001.
- Hebb DO. *The Organization of Behavior*. John Wiley & Sons; 1949.
- Heckerman D. *Probabilistic Similarity Networks*. MIT Press, Cambridge, Massachussets; 1991.
- Hopfield JJ. *Neurons with graded response have collective computational properties like those of two-state neurons*. Proceedings of the National Academy of Sciences of the United States of America, 79, 2554-2558; 1982.
- Minsky, M.; Papert, P.. *Perceptrons: An introduction to computational geometry*. MIT Press, Massachussets, 1969.

Referências Bibliográficas

- Mario MC. *Proposta de Aplicação das Redes Neurais Artificiais Paraconsistentes como Classificador de Sinais Utilizando Aproximação Funcional*. Univ. Federal de Uberlândia, Dissertação de Mestrado, Uberlândia; 2003.
- McCarthy J. *Programs with common sense*. In Proceedings of the Symposium on Mechanisation of Thought Processes, Vol. 1, pp. 77-84, London. Her Majesty's Stationery Office; 1958.
- McCulloch W, Pitts W. *A logical calculus of the ideas immanent in nervous activity*. Bulletin of Mathematical Biophysics 5, 115-133; 1943.
- Rosenblatt F. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan, Chicago; 1962.
- Rumelhart DE, McClelland JL. *Parallel Distributed Processing*. MIT Press, Cambridge, Massachusetts; 1986.
- Turing A. *Computing machinery and intelligence*. Mind, 59, 433-460; 1950.

Referências Bibliográficas

- Medeiros, L.F.. *Redes Neurais em Delphi*. Editora Visual Books. Florianópolis, 2003.
- Cybenko, G.. *Continuous valued neural networks with two hidden layers are sufficient*, Technical report, Department of Computer Science, Tufts University, 1988.
- Cybenko, G.. *Approximation by superpositions of sigmoid function*, *Mathematics of Control, Signals and Systems*, 2:303-314, 1989.